

FECAP

Curso de Ciência da Computação

Trabalho de Cálculo I

Sistema de Dia e Noite em Unity

Alunos: Polianny Santos da Silva

Letícia Rodrigues da Silva

Gian Carlos Lemes Vieira

Otavio Sanches Pierini

Eliaquim Marcelino

Professora: Renata Muniz Do Nascimento

Data: Abril de 2025

Sumário

1. Introdução	4
2. Objetivo do Script	4
3. Fundamentação Matemática	4
4. Estrutura e Funcionamento	5
5. Modificações e Configurações do Script	5
6. Considerações Finais	6
7. Código-Fonte	7

1. Introdução

No processo de criação de jogos digitais, a ambientação é um fator fulcral para proporcionar uma experiência imersiva ao jogador. Um método para melhorar essa dimensão é a recriação de ciclos naturais, como o dia e a noite. Com esse objetivo, nós, do grupo Code Crafters, desenvolvemos um script referido como “DiaNoite”, que tem a finalidade de implementar um sistema de passagem do tempo, utilizando cálculos matemáticos simples e recursos visuais da engine Unity.

O intuito deste texto é descrever o funcionamento do código, deixando clara a sua lógica e suas aplicações no jogo Urban Mind.

2. Objetivo do Script

O objetivo do script DiaNoite é implantar um sistema que reproduza o ciclo de luz do dia e da noite no ambiente do jogo, proporcionando mudanças de iluminação e tonalidade ao longo do tempo.

3. Fundamentação Matemática

Para a criação do sistema, foram utilizados conceitos básicos de tempo, organizados em relações matemáticas simples:

1 hora = 60 minutos

1 minuto = 60 segundos

Portanto: 1 hora = 3.600 segundos

E: 1 dia (24 horas) = 86.400 segundos

Esses valores foram utilizados como referência para representar a passagem do tempo no jogo. Contudo, como o tempo no ambiente virtual não precisa necessariamente refletir o tempo real, o código possibilita a modificação do comprimento de um "dia" no jogo de forma livre.

Ilustração de adaptação matemática: caso desejemos que um ciclo total (incluindo dia e noite) tenha a duração de 10 minutos na realidade, isso equivale a:

$10 \text{ minutos} \times 60 \text{ segundos} = 600 \text{ segundos}$

Esse valor será utilizado como base no script para determinar a relação de tempo e regular as outras variáveis relacionadas ao movimento do sol e à iluminação do ambiente.

4. Estrutura e Funcionamento

1. Atualização Contínua do Tempo

Dentro do método Update(), que acontece em cada frame, o tempo do jogo é continuamente ajustado de acordo com o tempo real que passou. Isso assegura um progresso constante e suave na simulação diária.

Cálculo matemático envolvido:

`tempoAtual += Time.deltaTime`

(exemplo: se o jogo roda a 60 fps, cada frame representa aproximadamente 0,016 segundos)

2. Cálculo da Percentagem do Dia

O script determina a porcentagem atual do ciclo diário, que oscila de 0 (começo do dia) a 1 (conclusão do dia). Essa porcentagem é calculada por:

`percentagemDoDia = tempoAtual / duracaoTotalDoCiclo`

Esse valor é utilizado para estabelecer como o sol e a iluminação se comportam na cena. Quando tempoAtual alcança duracaoTotalDoCiclo, ele se reinicia, dando início a um novo “dia”.

3. Simulação do Movimento Solar

A luz direcional da cena simboliza o Sol. Sua rotação em torno do eixo X é ajustada com base na fração do dia, gerando a ilusão do movimento solar.

Cálculo realizado de maneira matemática:

$\text{ânguloX} = \text{fraçãoDoDia} \times 360^\circ$

Assim, a luz desloca-se de forma constante pelo céu, mudando de lugar à medida que o tempo passa.

5. Modificações e Configurações do Script

O script “DiaNoite” foi criado para possibilitar a alteração do tempo do ciclo de dia e noite. A variável `duracaoDoDia` estabelece a duração total do ciclo (em segundos), e isso pode ser mudado conforme necessário para implementar um ciclo mais veloz ou mais prolongado, conforme a demanda do jogo.

Como modificar a duração do ciclo

A variável `duracaoDoDia` fornece ao desenvolvedor o poder de definir a duração total do ciclo de dia e noite, sendo a duração ajustada em segundos. Este valor é utilizado para determinar a proporção do dia e, com isso, ajustar a rotação da luz direcional.

6. Considerações Finais

O script `DiaNoite` demonstra como a integração de conceitos matemáticos básicos, combinados com a capacidade visual da Unity, pode resultar em uma mecânica fluida e imersiva.

O emprego prático de proporções, conversões de unidades (como segundos, minutos e horas) e interpolação de valores foi primordial para o funcionamento eficaz do sistema, evidenciando a relação inerente da matemática na elaboração de mecânicas de jogos digitais.

Este sistema não apenas enriquece a atmosfera visual do jogo, mas também propõe novas dinâmicas e desafios, variando conforme o horário do dia. O ciclo pode ser utilizado para impactar a dificuldade, a presença de personagens, a visibilidade e muito mais — todos esses aspectos sustentados por uma base matemática sólida.

7. Código

```
using System.Collections;

using System.Collections.Generic;

using TMPro;

using UnityEngine;

using System;

public class DiaNoite : MonoBehaviour

{

    [Header("Configurações de Tempo")]

    [SerializeField] private Transform luzDirecional;          // Referência à luz direcional (sol)

    [SerializeField] private int duracaoDoDia = 120;          // Duração de um dia no jogo em segundos

    [SerializeField] private TextMeshProUGUI horarioText; // Referência ao texto que exibe o horário na tela

    [Header("Céu e Lua")]

    [SerializeField] private Material skyboxDia                // Skybox usada durante o dia

    [SerializeField] private Material skyboxNoite;            // Skybox usada durante a noite

    [SerializeField] private GameObject lua;                  // Objeto da lua

    [SerializeField] private Transform luaTransform;           // Transform da lua (não está sendo usado diretamente aqui)

    [Header("Transição")]

    [SerializeField] private Light luzSol; // Luz que representa o sol (usada para transição)

    [SerializeField] private float intensidadeDia = 1.2f;      // Intensidade do sol durante o dia
```

```

[SerializeField] private float intensidadeNoite = 0.1f;           // Intensidade da lua durante a
noite

[SerializeField] private float velocidadeTransicao = 1f;          // Velocidade da transição de
luz

private float segundos;     // Segundos acumulados do dia atual

private float multiplicador; // Multiplicador para converter tempo real em tempo do jogo

private string[] nomesDasFases = { "Manhã", "Tarde", "Noite", "Madrugada" }; // Vetor com nomes
dos períodos do dia

void Start()
{
    multiplicador = 86400f / duracaoDoDia;           // Calcula quantos segundos de jogo se
passam por segundo real (86400s = 24h)
}

void Update()
{
    segundos += Time.deltaTime * multiplicador;      // Incrementa o tempo do jogo
baseado no tempo real

    if (segundos >= 86400f) segundos = 0f;           // Reinicia o contador ao completar 24h
    {
        GirarCeu(); // Atualiza a rotação do sol e da lua

        CalcularTempo();           // Atualiza o texto do horário e nome do período

        AtualizarDiaNoite();       // Troca o skybox, a lua e a intensidade da luz conforme o
horário

        TransicaoLuzes();           // Faz transição suave da intensidade da luz

```



```

    }
}

private void GirarCeu()
{
    float rotacaoX = Mathf.Lerp(-90f, 270f, segundos / 86400f);           // Gira o sol em 360°
    ao longo do dia

    luzDirecional.rotation = Quaternion.Euler(rotacaoX, 0f, 0f);           // Aplica a rotação
    calculada ao sol

    if (lua != null)
    {
        float rotacaoLua = Mathf.Lerp(90f, 450f, segundos / 86400f);       // Rotação da lua
        (oposta ao sol)

        Quaternion rotacao = Quaternion.Euler(rotacaoLua, 0f, 0f);           // Cria rotação da lua

        Vector3 direcaoLua = rotacao * Vector3.forward;                       // Direção que a lua deve
        apontar

        lua.transform.position = direcaoLua * -600f + new Vector3(0f, 0f, 0f); // Posiciona a
        lua no céu em oposição ao sol

        lua.transform.LookAt(Vector3.zero); // Faz a lua olhar para o centro da cena
    }
}

private void CalcularTempo()
{
    int hora = (int)(segundos / 3600f); // Converte os segundos acumulados em hora (0-
23)

```

```

string periodo = ""; // Variável que irá armazenar o nome do período

if (hora >= 6 && hora < 12)
    periodo = nomesDasFases[0]; // Manhã
else if (hora >= 12 && hora < 18)
    periodo = nomesDasFases[1]; // Tarde
else if (hora >= 18 && hora < 24)
    periodo = nomesDasFases[2]; // Noite
else
    periodo = nomesDasFases[3]; // Madrugada

string horaFormatada = TimeSpan.FromSeconds(segundos).ToString(@"hh\:mm"); // Converte
os segundos acumulados para o formato hh:mm

horarioText.text = horaFormatada + " - " + periodo; // Atualiza o texto com
horário e nome do período
}

private void AtualizarDiaNoite()
{
    int hora = (int)(segundos / 3600f); // Converte os segundos acumulados em hora (0-
23)

    bool ehDia = hora >= 6 && hora < 18; // Define se é dia com base na hora (entre 6h e 18h)

    if (ehDia)
        RenderSettings.skybox = skyboxDia; // Troca o skybox para dia
    else
        RenderSettings.skybox = skyboxNoite; // Troca o skybox para noite

```

```

    if (lua != null)

        lua.SetActive(!ehDia); // Ativa a lua apenas à noite


    if (ehDia)

        luzDirecional.GetComponent<Light>().intensity = 1.2f;        // Ajusta a intensidade do
sol durante o dia

    else

        luzDirecional.GetComponent<Light>().intensity = 0.2f;        // Ajusta a intensidade do
sol durante a noite

}


private void TransicaoLuzes()
{

    int hora = (int)(segundos / 3600f);        // Converte os segundos em hora

    bool ehDia = hora >= 6 && hora < 18;        // Verifica se é dia


    float alvoSol; // Intensidade alvo do sol if

    (ehDia)

        alvoSol = intensidadeDia;        // Define intensidade do sol para o dia

    else

        alvoSol = intensidadeNoite;        // Define intensidade do sol para a noite


    luzSol.intensity = Mathf.Lerp(luzSol.intensity, alvoSol, Time.deltaTime * velocidadeTransicao);
// Faz a transição suave da intensidade do sol

}

}

```

